

Verifiable Conjunctive Field Keyword Searchable Encryption With Aggregate Keys for E-Health Cloud

Xihui Lu¹, Zhenfu Cao¹, Senior Member, IEEE, Xiaolei Dong², Member, IEEE, and Jiachen Shen¹, Member, IEEE

Abstract—With the rapid proliferation of Internet of Things (IoT)-enabled healthcare systems, sensitive data, such as electronic health records (EHRs) are increasingly outsourced to semi-trusted e-health clouds, raising critical concerns about privacy and verifiability. This article investigates secure and efficient keyword-based data retrieval in such environments. Key-aggregate searchable encryption (KASE) enables a data owner to share multiple encrypted documents with users using a single compact key, but existing schemes offer limited resistance to offline keyword guessing attacks (KGA) and insufficient protection against keyword privacy leakage. To systematically expose broader vulnerabilities in existing KASE constructions, we present a novel and general attack, termed expanded inside trapdoor attack (EITA), which targets both the security of aggregate keys and the privacy of keywords in unauthorized documents. Furthermore, flexible search methods and result verification are essential for improving KASE performance. To this end, we propose a new KASE scheme that supports conjunctive queries and employs aggregate signatures for verification, while being able to resist KGA and EITA. The security analysis and performance evaluation both demonstrate that our proposed scheme is provably secure and practically efficient in IoT-enabled healthcare settings.

Index Terms—Conjunctive query, electronic health record (EHR), inside trapdoor attack (ITA), key-aggregate searchable encryption (KASE), verifiable.

I. INTRODUCTION

IN RECENT years, the rapid proliferation of Internet of Things (IoT)-enabled healthcare systems has led to an explosive growth in sensitive data, particularly electronic health records (EHRs), which are often outsourced to the semi-trusted e-health cloud for scalable storage and convenient access. While this paradigm significantly improves healthcare service delivery, it also introduces serious privacy and security concerns. Once data is outsourced, the data owner loses direct

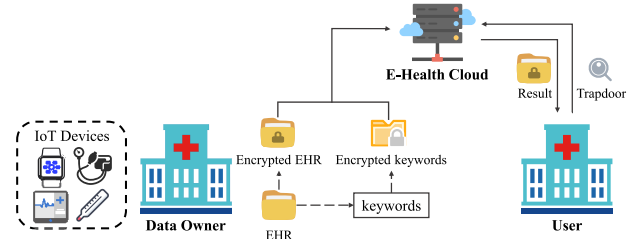


Fig. 1. SE architecture for e-health cloud.

control, which may lead to privacy leakage and a series of security risks. A typical solution is to encrypt before outsourcing [1], [2], [3]. Although encryption protects privacy, it renders data semantically opaque, complicating retrieval and posing practical challenges.

To search for data while safeguarding privacy, searchable encryption (SE) technology [4] has been introduced. This technology encrypts data and stores it in the cloud without exposing the plaintext, allowing users to conduct searches on the ciphertext. SE is mainly classified into symmetric SE (SSE) and public key encryption with keyword search (PEKS). SSE, proposed by Song et al. [4], employed a single key for simplicity and speed, but requires secure key distribution. In contrast, PEKS, proposed by Boneh et al. [5], employs a pair of public and private keys, eliminating the need for key distribution but increasing system complexity.

Given its ability to protect data privacy while enabling efficient keyword search, SE is particularly suitable for data sharing in IoT-based healthcare scenarios. For example, in collaborative diagnosis or medical data exchange, healthcare institutions often need to selectively share EHRs collected from various IoT devices (e.g., wearable monitors, smart sensors) with multiple authorized parties.

As illustrated in Fig. 1, a SE architecture allows encrypted data storage and keyword-based search in cloud-assisted healthcare systems, providing a basic framework for secure sharing.

Consider a hospital H managing patient records for $S = \{Alice, Bob, Carol, Dan, Ella\}$. Suppose H wants to share data subset $S_A = \{Alice, Bob, Ella\}$ with hospital A , and data subset $S_B = \{Bob, Carol, Dan, Ella\}$ with hospital B . In SSE, the number of keys distributed is proportional to the number of documents shared; thus, hospital H needs to distribute three keys to A and four keys to B . Furthermore, hospitals A and B must generate multiple search trapdoors for keywords of

Received 29 June 2025; revised 19 July 2025; accepted 9 August 2025. Date of publication 25 August 2025; date of current version 7 November 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2020YFA0712300; in part by the National Natural Science Foundation of China under Grant 62132005, and Grant 62172162; in part by the Shanghai Trusted Industry Internet Software Collaborative Innovation Center; and in part by the Fundamental Research Funds for the Central Universities. (Corresponding author: Zhenfu Cao; Jiachen Shen.)

The authors are with the Shanghai Collaborative Innovation Center of Trusted Industry Internet Software, East China Normal University, Shanghai 200062, China (e-mail: 51265902056@stu.ecnu.edu.cn; zfciao@sei.ecnu.edu.cn; dongxiaolei@sei.ecnu.edu.cn; jcsen@sei.ecnu.edu.cn).

Digital Object Identifier 10.1109/JIOT.2025.3602276

interest. In contrast, in PEKS, the number of encryptions for a document is proportional to the number of authorized users; thus, hospital H needs to encrypt the documents for “Bob” and “Ella” twice to accommodate the public keys of both A and B . Both methods become inefficient and cumbersome as the number of users and documents increases.

To solve the above problems, Cui et al. [6] proposed key-aggregate SE (KASE), which allows the data owner to share multiple encrypted documents via a single constant-size aggregate key. Authorized users can search over all shared documents using just one trapdoor. The aggregate key is extracted by the data owner from selective documents to be shared using his private key. In the example above, hospital H would only need to generate and send one aggregate key to each of A and B , significantly reducing computational, communication, and storage overhead while enhancing scalability in IoT-enabled settings.

However, previous KASE schemes are typically vulnerable to offline keyword guessing attacks (KGA) [7], which extract the embedded keyword from a given trapdoor using a brute force method. Zhou et al. [8] presented a new attack on Cui’s scheme [6], termed the inside trapdoor attack (ITA) by Liu et al. [9]. The attack allows an attacker inside the system to guess the aggregate keys of users who share access to certain documents. Many existing KASE schemes are not resistant to ITA. While Liu et al. proved their scheme is secure against ITA, their security argument is largely heuristic and lacks a formal proof. Moreover, we have found that an insider can still obtain keyword information about unauthorized documents by forging trapdoors. Reference [9] also defines a security model to capture ciphertext privacy and allows conjunctive queries for finer-grained results and higher efficiency, enabling users to submit a single trapdoor for multiple keywords of interest. Reference [10] focuses on both ciphertext and trapdoor privacy but only considers external attackers regarding trapdoors. Neither [9] nor [10] implements ciphertext indistinguishability and trapdoor indistinguishability, which are essential for establishing computational security under the SE scheme.

Moreover, man-in-the-middle (MITM) attacks on the public channel can compromise the integrity of the search results returned by the server. Verification allows users to ensure the integrity of search results, thereby maintaining the accuracy and reliability of data. Similar to aggregate keys, using aggregate signatures [11] for verification is valuable for reducing overhead.

To address the above issues, we propose a novel KASE scheme called verifiable conjunctive field keyword SE with aggregate keys (VC-KASE), and present a novel attack covering a broader range of vulnerabilities. In addition, two security models are defined to capture the privacy of ciphertext and trapdoor, respectively. Finally, we support conjunctive queries for more flexible searches and provide verification to maintain the integrity of search results. The main contributions of our work are as follows.

- 1) *Expanded ITA (EITA)*: We present a novel attack with a broader scope, called EITA, against Liu et al.’s KASE-CQ scheme. EITA extends the ITA by not only targeting the security of aggregate keys but also focusing on

the privacy of keywords in unauthorized documents. To formalize this enhanced threat, we define a new security model—document security against expanded trapdoor attack (DOC-EITA) and formally prove the security of our VC-KASE scheme under this model.

- 2) *Ciphertext Indistinguishability and Trapdoor Indistinguishability*: We formally define two security models: ciphertext indistinguishability against chosen keyword attack (CT-IND-CKA) and trapdoor indistinguishability against chosen keyword attack (TR-IND-CKA), which correspond to ciphertext and trapdoor privacy, respectively. Notably, trapdoor indistinguishability has the ability to resist offline KGA [12]. We then prove the security of our VC-KASE scheme based on these definitions.
- 3) *Conjunctive Query and Efficient Verification*: To ensure search flexibility and result integrity while minimizing overhead, we support keyword conjunction in queries and use aggregate signatures for verification. Users can search multiple keywords with a single trapdoor, and the cloud server returns only one aggregate signature, regardless of the number of results.
- 4) *Performance Evaluation*: Based on KASE primitive, we construct a keyword SE scheme with conjunctive query and verification. Performance evaluations demonstrate the efficiency and practicality of our VC-KASE scheme in IoT-enabled healthcare scenarios.

The paper is organized as follows. In Sections II and III, we introduce the related works and preliminaries, respectively. Section IV formalizes the system model, threat model, system framework, and security models. Section V describes our proposed scheme in detail. Sections VI and VII present the security and performance evaluations of our proposed scheme, respectively. Finally, Section VIII concludes this article.

II. RELATED WORKS

In this section, we review previous works on “public key authenticated encryption with keyword search (PAEKS)” aimed at resisting KGA and “KASE.”

A. PAEKS

Standard PEKS schemes [13], [14], [15] are inherently vulnerable to KGA. Rhee et al. [12] introduced the concept of “trapdoor indistinguishability” and demonstrated that it is a sufficient condition to resist KGA. Huang and Li [16] proposed the first effective PAEKS scheme to resist KGA, which embeds the sender’s private key in the encrypted data to ensure that only the sender can generate it. Qin et al. [17] revised [16] to achieve full ciphertext indistinguishability. Chen et al. [18] proposed a new dual-server PEKS framework to resist KGA initiated by the malicious server. Cheng and Meng [19] proposed the first lattice-based PAEKS schemes to prevent quantum attacks [20]. In addition to these, more variants of PAEKS schemes [21], [22], [23], [24], [25] that defend against KGA have been proposed.

More recently, several works have focused on enhancing the efficiency and practicality of PAEKS in real-world applications. For example, Lu and Li [26] proposed a lightweight scheme that eliminates bilinear pairings and strengthens the security model to resist adaptively chosen target attacks, making it suitable for mobile devices. In the context of cloud-based commercial services, Chen et al. [27] presented PAESS, which enables similar keyword search under a pay-per-query model, while supporting billing verifiability and lightweight computation. To address scalability and support secure data sharing among multiple users, Xu et al. [28] designed a multiuser PAEKS scheme that maintains constant ciphertext and trapdoor sizes, and introduces a dedicated receiver server to prevent inside KGA. These efforts reflect a shift toward more efficient and application-oriented PAEKS designs.

B. KASE

Chu et al. [29] proposed a new public-key cryptosystem called key-aggregate cryptosystem (KAC). Since then, many KAC schemes have been proposed [30], [31], [32], [33]. The technology is capable of combining any set of keys into a single key that contains the energy of all combined keys, but does not provide the ability to search for encrypted data. Cui et al. [6] proposed an effective KASE scheme that is able to authorize an arbitrary number of documents using a single aggregate key and allows users to generate a trapdoor to retrieve these documents. However, they did not formally define security models for their scheme. Subsequent KASE schemes [34], [35], [36], [37], [38] also lack formal security models. Kiayias et al. [39] noted that Cui's scheme is vulnerable to cross pairing attacks, which can lead to ciphertext privacy leakage.

In 2018, Zhou et al. [8] presented a new attack on Cui's scheme [6], which was referred to as the ITA by Liu et al. [9]. An attacker inside the system can guess the aggregate keys of users who share access to certain documents under this attack. None of these schemes [34], [35], [36], [40], [41], [42], [43] effectively resist the ITA since they are constructed similarly to [6]. Liu et al. proved their KASE-CQ scheme [9] is secure against this attack, but we have found that an insider can still obtain keyword information about unauthorized documents by forging trapdoors.

The schemes [8], [9], [10], [40], [41], [44], [45] give the formal security proof to their schemes. Notably, [9], [41] focus only on ciphertext indistinguishability. [10] only limits to external attackers in the proof of trapdoor security. In the schemes [9], [40], [41], [44], we found that an attacker can extract keyword from ciphertexts and trapdoors by cross pairing them with public or allowed query values, making these schemes vulnerable to offline KGA. In contrast, [8], [45] resist offline KGA by setting the number of keys related to the number of documents. However, a common limitation of the security models in [8], [9], [40], [41], [44], [45] is their lack of comprehensiveness, as they do not include formal security proofs against attacks from the server.

To the best of our knowledge, the KASE-CQ scheme proposed by [9] is currently the only KASE scheme that

TABLE I
COMPREHENSIVE COMPARISON

	vaKASE [34]	dKASE [10]	KASE-CQ [9]	VC-KASE
Against EITA	×	×	×	✓
Cipher-ind	×	✓	×	✓
Trap-ind	×	×	×	✓
Conjunctive Query	×	×	✓	✓
Verifiable	✓	×	×	✓

Cipher-ind: Ciphertext indistinguishability;
Trap-ind: Trapdoor indistinguishability.

TABLE II
NOTATIONS

Notation	Description
n	maximum number of documents owned by a data owner
m	number of keyword fields per document
l	number of queried keywords
d	number of matched documents
$params$	system global parameters
pk_o/sk_o	public/private key of the data owner
pk_s/sk_s	public/private key of the cloud server
i	document identifier
W_i	keyword set of document i
cf_i	ciphertext of document i
CT_i	keyword ciphertext of document i
σ_i	signature of document i
σ	aggregate signature
$ S $	number of elements in set S
S_A	document set authorized for A
K_{agg}^S	aggregate key of document set S
Q	queried keyword set
Tr	aggregate trapdoor
Tr_i	trapdoor of document i

truly supports conjunctive queries. In contrast, the schemes in [42], [43], [44] require the query keyword set to exactly match the encrypted keyword set of each document, effectively reducing the search to exact-set matching rather than supporting flexible conjunctive queries. Li et al. [46] proposed a verifiable KASE scheme based on bloom filter (BF), but returned search results can be forged by the server. Li et al. [41] combined BF with Merkle hash tree to achieve verification. Liu and Liu [34] proposed a verifiable and authenticated KASE scheme (denoted as vaKASE), which requires two rounds of interaction during the verification process. Liu et al. [10] used aggregate signatures to achieve verification in their multiowner setting scheme. However, the limitation of single keyword queries necessitates multiple verifications when querying multiple keywords. Yao et al. [47] proposed a lattice-based KASE scheme to prevent quantum attacks.

Table I shows a comprehensive comparison of our VC-KASE scheme with other relevant works.

III. PRELIMINARIES

A. Notations

The necessary notations are provided in Table II.

B. Bilinear Pairing

Let $\mathcal{G}$ and $\mathcal{G}_T$ be two cyclic multiplicative groups of prime order p , and g is a generator of $\mathcal{G}$. A symmetric bilinear pairing is a map $e : \mathcal{G} \times \mathcal{G} \rightarrow \mathcal{G}_T$ satisfying properties as follows.

- 1) *Bilinearity*: $\forall u, v \in \mathcal{G}$ and $a, b \in \mathbb{Z}_p^*$, $e(u^a, v^b) = e(u, v)^{ab}$.
- 2) *Nondegeneracy*: $\exists u, v \in \mathcal{G}$, $e(u, v) \neq 1$.
- 3) *Computability*: $\forall u, v \in \mathcal{G}$, there is an efficient algorithm to compute $e(u, v)$.

C. Hardness Assumptions

1) *Computational Diffie–Hellman Assumption*: Computational Diffie–Hellman (CDH) problem: Given a 3-tuple $(g, g^a, g^b) \in \mathcal{G}^3$, where a, b are randomly chosen from $\mathbb{Z}_p$, compute g^{ab} .

Definition 1: The CDH assumption holds in $\mathcal{G}$ if for any probabilistic polynomial time (PPT) algorithm $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that

$$\Pr[\mathcal{A}(g, g^a, g^b) = g^{ab}] \leq \text{negl}(\kappa).$$

2) *Divisible CDH Assumption*: Divisible CDH problem: Given a 3-tuple $(g, g^a, g^b) \in \mathcal{G}^3$, where a, b are randomly chosen from $\mathbb{Z}_p$, compute $g^{(b/a)}$.

Definition 2: The divisible CDH assumption holds in $\mathcal{G}$ if for any PPT algorithm $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that

$$\Pr[\mathcal{A}(g, g^a, g^b) = g^{b/a}] \leq \text{negl}(\kappa).$$

3) *Discrete Logarithm Assumption*: discrete logarithm (DL) problem: Given a 2-tuple $(g, g^a) \in \mathcal{G}^2$, where a is randomly chosen from $\mathbb{Z}_p$, compute a .

Definition 3: The DL assumption holds in $\mathcal{G}$ if for any PPT algorithm $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that

$$\Pr[\mathcal{A}(g, g^a) = a] \leq \text{negl}(\kappa).$$

4) *n Diffie–Hellman Exponent Assumption*: n Diffie–Hellman exponent (n-DHE) problem: Given a $2n$ -tuple $(g, g^{\alpha}, g^{\alpha^2}, \dots, g^{\alpha^n}, g^{\alpha^{n+2}}, \dots, g^{\alpha^{2n}}) \in \mathcal{G}^{2n}$, compute $g^{\alpha^{n+1}}$. For convenience, we use g_i to denote g^{α^i} .

Definition 4: The n-DHE assumption holds if for any PPT algorithm $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that

$$\Pr[\mathcal{A}(g, g_1, g_2, \dots, g_n, g_{n+2}, \dots, g_{2n}) = g_{n+1}] \leq \text{negl}(\kappa).$$

IV. MODEL DEFINITION

A. System Model

As shown in Fig. 2, our VC-KASE consists of three types of entities: a data owner, users and a cloud server. The roles of these entities are described below.

- 1) *Data Owner*: The entity is responsible for generating system parameters, categorizing documents into classes, encrypting them with unique keys, and outsourcing the encrypted data and signatures to the cloud server. It also generates aggregate keys for the shared document dataset and securely transmits them to users.
- 2) *Users*: Upon receiving the aggregate key, this entity can issue conjunctive search queries based on its keywords of interest using the aggregate trapdoor generated. The entity also verifies the authenticity of the returned results with the aggregate signature following a search.

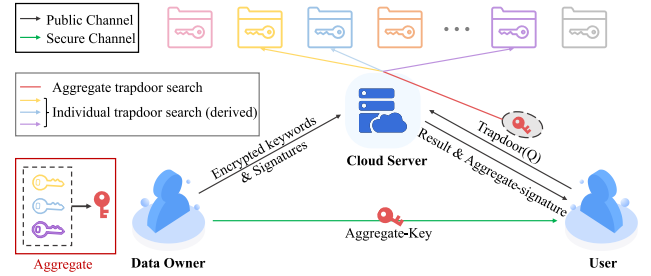


Fig. 2. System model of VC-KASE.

- 3) *Cloud Server*: The entity stores ciphertexts and signatures from the data owner. Upon receiving the aggregate trapdoor from the user, the entity adjusts it for each document class, performs the search, and returns the matched documents along with the aggregate signature generated to the user.

B. Threat Model

In our threat model, we consider three types of entities.

- 1) *Data Owner*: Fully trusted. The data owner strictly follows the protocol and does not attempt to leak, misuse, or infer any private information.
- 2) *Cloud Server*: Considered semi-honest (honest-but-curious). It executes all protocols faithfully, but may attempt to infer confidential information from the data it processes.
- 3) *Users*: Considered malicious. Users may collude with each other by sharing their aggregate keys or other knowledge in order to maximize unauthorized access to encrypted documents.

We also assume that the cloud server and users do not collude with each other.

C. VC-KASE Framework

The main algorithms of our VC-KASE are as follows.

- 1) **Setup**($1^\kappa, n$): Given a security parameter κ , the data owner returns the public parameter $params$.
- 2) **KeyGen_o**: The data owner outputs its public-private key pair (pk_o, sk_o) .
- 3) **KeyGen_s**: The cloud server outputs its public-private key pair (pk_s, sk_s) .
- 4) **Encrypt**($pk_o, sk_o, pk_s, i, W_i, cfi$): Input document identifier i , keyword set W_i and document ciphertext cfi , the data owner returns the keyword ciphertext CT_i and signature σ_i .
- 5) **Extract**(sk_o, S): Input a document subset S , the data owner returns the aggregate key K_{agg}^S and then transmits it to the user via a secure channel.
- 6) **Trapdoor**(pk_o, K_{agg}^S, Q): Input the queried keyword set Q , the user returns the aggregate trapdoor Tr for all documents associated to K_{agg}^S .
- 7) **Test**($Tr, S, i, CT_i, sk_s, cfi, \sigma_i$): The cloud server first initializes two sets Σ and C . Input the trapdoor Tr and set S , the cloud server adjusts Tr to the trapdoor Tr_i of the document with identifier i belonging to S . Then, input

CT_i , if the queried keywords is the same as CT_i in the corresponding fields, the cloud server returns 1 and adds σ_i to Σ and $\{i, cf_i\}$ to C , otherwise returns 0.

- 8) **Verify**(pk_o, σ, C): Input the aggregate signature σ and document identifier-ciphertext set C , the user returns 1 if C is a valid document identifier-ciphertext set and σ corresponds to it, otherwise returns 0.

D. EITA Definition

In 2018, Zhou et al. [8] presented a new attack on Cui's scheme [6], termed ITA by Liu et al. [9]. The attack allows an attacker inside the system to guess the aggregate keys of users who share access to certain documents. Although Liu et al. claimed that their KASE-CQ scheme is secure against ITA, the argument is heuristic and lacks a formal proof. More importantly, we identify that KASE-CQ remains vulnerable to keyword inference attacks through trapdoor forgery.

We further present EITA, a stronger attack model, where an insider can exploit access to shared documents to infer the aggregate keys of other users or keyword information about unauthorized documents belonging to those users. Specifically, an attacker can use knowledge of the aggregate key to forge a trapdoor that infers keyword information about unauthorized documents, even if he cannot access them by other methods. Unlike ITA, which targets only aggregate key inference, EITA explicitly links the inference of keyword information directly to the manipulation of aggregate keys, covering a broader range of vulnerabilities. It is capable of compromising a wide range of existing KASE schemes [9], [10], [34], [35], [36], [40], [41], [42], [43], including those that have been claimed or proven secure against ITA. This reveals a systemic design flaw in current KASE constructions.

To demonstrate the practicality of EITA, we analyze KASE-CQ as a case study.

We begin by briefly recall the key procedures of KASE-CQ relevant to the attack:

- 1) **KeyGen**: Randomly pick $\beta, \gamma \in Z_p^*$ and output the public-private key pair (pk, sk) , where $pk = (v_1 = g^\beta, v_2 = g^\gamma)$ and $sk = (\beta, \gamma)$.
- 2) **Extract**(sk, S): Output the aggregate key $K_{agg}^S = \prod_{j \in S} s_{n+1-j}^\gamma$ related to document subset $S \subseteq \{1, \dots, n\}$ and securely send it to the user.
- 3) **Trapdoor**(pk, K_{agg}^S, Q): Input the queried keyword set Q , randomly pick $s \in Z_p^*$ and output the trapdoor $Tr = \{T_1, T_2, T_3\} = \{K_{agg}^S \cdot g^s, [v_1^s / g^{\sum_{k=1}^l H(w'_{jk})}], \{j_k\}_{k=1}^l\}$, where T_3 is the keyword field set of Q .

Now suppose an insider adversary $\mathcal{A}$ holds an aggregate key K_{agg}^S for a document set S such that $i^* \notin S$. $\mathcal{A}$ can perform the EITA attack as follows.

- 1) *Step 1*: $\mathcal{A}$ eavesdrops on a trapdoor submission

$$Tr = (T_1, T_2, T_3) = \left(K_{agg}^S \cdot g^s, \frac{v_1^s}{g^{\sum_{k=1}^l H(w'_{jk})}}, \{j_k\}_{k=1}^l \right)$$

where $i^* \in S^*$, and S' is the result set returned by the cloud. Suppose $i \in S \cap S'$ and $i^* \notin S'$.

- 2) *Step 2*: $\mathcal{A}$ leverages its authorized access to document $i \in S$ to derive the keyword set $Q = \{w_{j_1}, w_{j_2}, \dots, w_{j_l}\}$ corresponding to the trapdoor Tr .
- 3) *Step 3*: $\mathcal{A}$ launch one of the following attacks.
 - a) $\mathcal{A}$ aims to infer a keyword w^* contained in the document i^* . To mount the attack, $\mathcal{A}$ forges a valid trapdoor Tr^* for the keyword w^* and the document identifier i^*

$$Tr^* = (T_1, T_2^*, T_3^*) \\ = \left(K_{agg}^{S^*} \cdot g^s, \frac{T_2}{g^{H(w^*)}} \cdot g^{\sum_{k=1}^l H(w_{j_k})}, \{j_k\}_{k=1}^l \right).$$

where T_1 is reused and T_3^* contains the index of w^* . $\mathcal{A}$ successfully forge a valid trapdoor Tr^* . If submitting Tr^* to the cloud yields a result set containing i^* , then w^* is confirmed to be associated with i^* .

- b) $\mathcal{A}$ aims to recover the aggregate key $K_{agg}^{S^*}$ for a document set $S^*(i^* \in S^*)$, where $K_{agg}^{S^*}$ is inferred from the trapdoor Tr .

From T_2 , $\mathcal{A}$ computes to get $v_1^s = g^{\beta s} = [v_1^s / g^{\sum_{k=1}^l H(w_{j_k})}] \cdot g^{\sum_{k=1}^l H(w_{j_k})}$. Given that $T_1 = K_{agg}^{S^*} \cdot g^s$, $\mathcal{A}$ can compute

$$K_{agg}^{S^*} = \frac{T_1}{g^s}.$$

$\mathcal{A}$ needs to solve the divisible CDH problem to get g^s from $v_1 = g^\beta$ and $g^{\beta s} = v_1^s$.

In summary, although recovering $K_{agg}^{S^*}$ is computationally hard under the divisible CDH assumption, EITA still allows an insider to infer keyword information about unauthorized documents by forging valid trapdoors. This demonstrates that KASE-CQ does not provide strong insider resistance under our enhanced attack model.

E. Security Model

Based on the threat model of our VC-KASE, four security properties are considered as follows.

- 1) *Ciphertext Privacy*: the keyword ciphertext does not disclose any information about the relative keyword set to unauthorized attackers.
- 2) *Trapdoor Privacy*: the keyword trapdoor does not disclose any information about the relative keyword set to unauthorized attackers.
- 3) *Aggregate Key Unforgeability*: attackers cannot forge a valid aggregate key for a document set containing unauthorized documents from the known aggregate keys.
- 4) *Aggregate Key and Keyword Privacy Against EITA*: Insider attackers cannot obtain the aggregate key or any unauthorized document keyword information by exploiting the eavesdropping trapdoor.

We perform the CT-IND-CKA, TR-IND-CKA, existential unforgeability of aggregate key against chosen keyword attack (EU-AK-CKA) and DOC-EITA to verify ciphertext privacy, trapdoor privacy, aggregate unforgeability, and aggregate key and keyword privacy against EITA, respectively. Among them, the TR-IND-CKA security model captures offline KGA.

These proofs use the standard model (SM), which offers stronger, realistic security guarantees, unlike the random oracle model (ROM), which relies on impractical idealized hash functions.

1) *CT-IND-CKA*: Suppose $\mathcal{A}_i (i = 1, 2)$ is a PPT adversary. We define the two CT-IND-CKA games as follows.

Game CT-1: Suppose $\mathcal{A}_1$ is a cloud server.

Setup: $\mathcal{A}_1$ submits set $S^* \subseteq [1, n]$ to challenge. $\mathcal{B}$ randomly picks $i^* \in S^*$ and sends it to $\mathcal{A}_1$. $\mathcal{B}$ runs *Setup* and *KeyGen_o*, *KeyGen_s* algorithms to generate *params*, (pk_o, sk_o) , (pk_s, sk_s) , then sends *params*, pk_o , (pk_s, sk_s) to $\mathcal{A}_1$.

Phase 1: $\mathcal{A}_1$ issues the following queries adaptively.

- 1) *Ciphertext Query*(i, W): Given a document identifier i and the related keyword set W , $\mathcal{B}$ runs *Encrypt* to generate the corresponding keyword ciphertext CT_i and returns it to $\mathcal{A}_1$.
- 2) *Trapdoor Query*(Q): Given a queried keyword set Q , $\mathcal{B}$ runs *Trapdoor* to generate the aggregate trapdoor Tr related to $K_{agg}^{S^*}$ and returns it to $\mathcal{A}_1$.

Challenge: $\mathcal{A}_1$ submits two distinct challenge keyword sets $W_0^*, W_1^* (|W_0^*| = |W_1^*|)$ to $\mathcal{B}$. Then $\mathcal{B}$ picks a random bit $c \in \{0, 1\}$ and runs *Encrypt* to generate the challenge ciphertext CT^* associated with the document identifier i^* , and returns it to $\mathcal{A}_1$. For document identifier i^* , W_0^* and W_1^* cannot have been queried. Moreover, W_0^* and W_1^* are not equal to the queried Q .

Phase 2: $\mathcal{A}_1$ issues ciphertext queries and trapdoor queries as in Phase 1. The limit is that $(i, W) \neq (i^*, W_0^*)$, $(i, W) \neq (i^*, W_1^*)$ for ciphertext query(i, W) and $Q \neq W_0^*$, $Q \neq W_1^*$ for trapdoor query(Q).

Guess: $\mathcal{A}_1$ returns $c' \in \{0, 1\}$ and wins if $c' = c$.

We define the advantage of $\mathcal{A}_1$ winning Game CT-1 as

$$\text{Adv}_{\mathcal{A}_1}^{\text{CT-IND-CKA}}(\kappa) = \left| \Pr[c' = c] - \frac{1}{2} \right|.$$

Game CT-2: Suppose $\mathcal{A}_2$ is a user.

Setup: $\mathcal{A}_2$ submits a document identifier $i^* (i^* \in [1, n])$ to challenge. $\mathcal{B}$ runs *Setup* and *KeyGen_o*, *KeyGen_s* algorithms to generate *params*, (pk_o, sk_o) , (pk_s, sk_s) , then sends *params*, pk_o , pk_s to $\mathcal{A}_2$.

Phase 1: $\mathcal{A}_2$ issues the following queries adaptively.

- 1) *Aggregate Key Query*(S): Given a document identifier set $S (i^* \notin S)$, $\mathcal{B}$ runs *Extract* to generate the aggregate key K_{agg}^S and returns it to $\mathcal{A}_2$.
- 2) *Ciphertext Query*(i, W): Given a document identifier i and the related keyword set W , $\mathcal{B}$ runs *Encrypt* to generate the corresponding keyword ciphertext CT_i and returns it to $\mathcal{A}_2$.

Challenge: $\mathcal{A}_2$ submits two distinct challenge keyword sets $W_0^*, W_1^* (|W_0^*| = |W_1^*|)$ to $\mathcal{B}$. Then $\mathcal{B}$ picks a random bit $c \in \{0, 1\}$ and runs *Encrypt* to generate the challenge ciphertext CT^* associated with the document identifier i^* , and returns it to $\mathcal{A}_2$. For document identifier i^* , W_0^* and W_1^* cannot have been queried.

Phase 2: $\mathcal{A}_2$ issues aggregate key queries and ciphertext queries as in Phase 1. The limit is that $(i, W) \neq (i^*, W_0^*)$ and $(i, W) \neq (i^*, W_1^*)$ for ciphertext query(i, W).

Guess: $\mathcal{A}_2$ returns $c' \in \{0, 1\}$ and wins if $c' = c$.

We define the advantage of $\mathcal{A}_2$ winning Game CT-2 as

$$\text{Adv}_{\mathcal{A}_2}^{\text{CT-IND-CKA}}(\kappa) = \left| \Pr[c' = c] - \frac{1}{2} \right|.$$

Definition 5: If $\text{Adv}_{\mathcal{A}_i}^{\text{CT-IND-CKA}}(\kappa)$ is negligible in κ for any PPT adversary, where i is either 1 or 2, the VC-KASE scheme is CT-IND-CKA secure.

2) *TR-IND-CKA*: Suppose $\mathcal{A}_i (i = 1, 2)$ is a PPT adversary. We define the two TR-IND-CKA games as follows.

Game TR-1: Suppose $\mathcal{A}_1$ is a cloud server.

Setup: $\mathcal{A}_1$ submits set $S^* \subseteq [1, n]$ to challenge. $\mathcal{B}$ randomly picks $i^* \in S^*$ and sends it to $\mathcal{A}_1$. $\mathcal{B}$ runs *Setup* and *KeyGen_o*, *KeyGen_s* algorithms to generate *params*, (pk_o, sk_o) , (pk_s, sk_s) , then sends *params*, pk_o , (pk_s, sk_s) to $\mathcal{A}_1$.

Phase 1: $\mathcal{A}_1$ issues the following queries adaptively.

- 1) *Ciphertext Query*(i, W): Given a document identifier i and the related keyword set W , $\mathcal{B}$ runs *Encrypt* to generate the corresponding keyword ciphertext CT_i and returns it to $\mathcal{A}_1$.
- 2) *Trapdoor Query*(Q): Given a queried keyword set Q , $\mathcal{B}$ runs *Trapdoor* to generate the aggregate trapdoor Tr related to $K_{agg}^{S^*}$ and returns it to $\mathcal{A}_1$.

Challenge: $\mathcal{A}_1$ submits two distinct challenge keyword sets $Q_0^*, Q_1^* (|Q_0^*| = |Q_1^*|)$ in the same keyword fields to $\mathcal{B}$. Then $\mathcal{B}$ picks a random bit $c \in \{0, 1\}$ and runs *Trapdoor* to generate the challenge trapdoor Tr^* associated with the document identifier i^* , and returns it to $\mathcal{A}_1$. Q_0^* and Q_1^* cannot have been queried. Moreover, Q_0^* and Q_1^* is not equal to the queried W associated with the document identifier i^* .

Phase 2: $\mathcal{A}_1$ issues ciphertext queries and trapdoor queries as in Phase 1. The limit is that $(i, W) \neq (i^*, Q_0^*)$, $(i, W) \neq (i^*, Q_1^*)$ for ciphertext query(i, W) and $Q \neq Q_0^*$, $Q \neq Q_1^*$ for trapdoor query(Q).

Guess: $\mathcal{A}_1$ returns $c' \in \{0, 1\}$ and wins if $c' = c$.

We define the advantage of $\mathcal{A}_1$ winning Game TR-1 as

$$\text{Adv}_{\mathcal{A}_1}^{\text{TR-IND-CKA}}(\kappa) = \left| \Pr[c' = c] - \frac{1}{2} \right|.$$

Game TR-2: Suppose $\mathcal{A}_2$ is a user.

Setup: $\mathcal{A}_2$ submits a document identifier $i^* (i^* \in [1, n])$ to challenge. $\mathcal{B}$ runs *Setup* and *KeyGen_o*, *KeyGen_s* algorithms to generate *params*, (pk_o, sk_o) , (pk_s, sk_s) , then sends *params*, pk_o , pk_s to $\mathcal{A}_2$.

Phase 1: $\mathcal{A}_2$ issues the following queries adaptively.

- 1) *Aggregate Key Query*(S): Given a document identifier set $S (i^* \notin S)$, $\mathcal{B}$ runs *Extract* to generate the aggregate key K_{agg}^S and returns it to $\mathcal{A}_2$.
- 2) *Ciphertext Query*(i, W): Given a document identifier i and the related keyword set W , $\mathcal{B}$ runs *Encrypt* to generate the corresponding keyword ciphertext CT_i and returns it to $\mathcal{A}_2$.

Challenge: $\mathcal{A}_2$ submits two distinct challenge keyword sets $Q_0^*, Q_1^* (|Q_0^*| = |Q_1^*|)$ in the same keyword fields to $\mathcal{B}$. Then $\mathcal{B}$ picks a random bit $c \in \{0, 1\}$ and runs *Trapdoor* to generate the challenge trapdoor Tr^* associated with the document identifier i^* , and returns it to $\mathcal{A}_2$.

Phase 2: $\mathcal{A}_2$ issues aggregate key queries and ciphertext queries as in Phase 1.

Guess: $\mathcal{A}_2$ returns $c' \in \{0, 1\}$ and wins if $c' = c$.

We define the advantage of $\mathcal{A}_2$ winning Game TR-2 as

$$\text{Adv}_{\mathcal{A}_2}^{\text{TR-IND-CKA}}(\kappa) = \left| \Pr[c' = c] - \frac{1}{2} \right|.$$

Definition 6: If $\text{Adv}_{\mathcal{A}_i}^{\text{TR-IND-CKA}}(\kappa)$ is negligible in κ for any PPT adversary, where i is either 1 or 2, the VC-KASE scheme is TR-IND-CKA secure. Additionally, the data owner's private keys are embedded into the encrypted data, which ensures the attacker cannot generate valid ciphertext for matching, thereby resisting offline KGA.

3) *EU-AK-CKA:* Suppose $\mathcal{A}$ is a PPT adversary. We define the EU-AK-CKA game as follows.

Setup: $\mathcal{A}$ submits a document identifier i^* ($i^* \in [1, n]$) to challenge. $\mathcal{B}$ runs *Setup* and *KeyGen_o*, *KeyGen_s* algorithms to generate *params*, (pk_o, sk_o) , (pk_s, sk_s) , then sends *params*, pk_o , pk_s to $\mathcal{A}$.

Phase 1: $\mathcal{A}$ issues the following queries adaptively.

- 1) *Aggregate Key Query*(S): Given a document identifier set S ($i^* \notin S$), $\mathcal{B}$ runs *Extract* to generate the aggregate key K_{agg}^S and returns it to $\mathcal{A}$.
- 2) *Ciphertext Query*(i, W): Given a document identifier i and the related keyword set W , $\mathcal{B}$ runs *Encrypt* to generate the corresponding keyword ciphertext CT_i and returns it to $\mathcal{A}$.

Forge: $\mathcal{A}$ returns an aggregate key $K_{\text{agg}}^{S^*}$ and wins if $K_{\text{agg}}^{S^*}$ is a valid aggregate key of document identifier set S^* with $i^* \in S^*$.

We define the advantage of $\mathcal{A}$ winning the EU-AK-CKA game as $\text{Adv}_{\mathcal{A}}^{\text{EU-AK-CKA}}(\kappa)$.

Definition 7: If $\text{Adv}_{\mathcal{A}}^{\text{EU-AK-CKA}}(\kappa)$ is negligible in κ for any PPT adversary, the VC-KASE scheme is EU-AK-CKA secure.

4) *DOC-EITA:* Suppose $\mathcal{A}$ is a PPT adversary. We define the DOC-EITA game as follows.

Setup: $\mathcal{A}$ submits a document identifier i^* ($i^* \in [1, n]$) to challenge. $\mathcal{B}$ runs *Setup* and *KeyGen_o*, *KeyGen_s* algorithms to generate *params*, (pk_o, sk_o) , (pk_s, sk_s) , then sends *params*, pk_o , pk_s to $\mathcal{A}$.

Phase 1: $\mathcal{A}$ issues the following queries adaptively.

- 1) *Aggregate Key Query*(S): Given a document identifier set S ($i^* \notin S$), $\mathcal{B}$ runs *Extract* to generate the aggregate key K_{agg}^S and returns it to $\mathcal{A}$.
- 2) *Trapdoor Query*(S^*): Given a document identifier set S^* ($i^* \in S^*$), $\mathcal{B}$ picks a queried keyword set Q and runs *Trapdoor* to generate the aggregate trapdoor Tr related to $K_{\text{agg}}^{S^*}$ and returns it to $\mathcal{A}$. The trapdoor Tr corresponds to a result set $S'(i^* \notin S')$.

Guess: The adversary $\mathcal{A}$ is allowed a limited number of attempts and wins if either of the following conditions is satisfied.

- 1) $\mathcal{A}$ outputs a keyword w^* contained in the keyword set of the challenge document i^* , where w^* is inferred by forging and submitting a valid trapdoor constructed from previously queried trapdoors.

- 2) $\mathcal{A}$ outputs an aggregate key $K_{\text{agg}}^{S^*}$ for a document set S^* with $i^* \in S^*$, where $K_{\text{agg}}^{S^*}$ is derived from a previously queried trapdoor.

We define the advantage of $\mathcal{A}$ winning the DOC-EITA game as the probability of successfully achieving either of the above goals. Let E_1 denote the event that $\mathcal{A}$ correctly outputs such a w^* , and E_2 denote the event that it successfully outputs such a $K_{\text{agg}}^{S^*}$. Then, the advantage of $\mathcal{A}$ is defined as

$$\text{Adv}_{\mathcal{A}}^{\text{DOC-EITA}}(\kappa) = \Pr[E_1 \vee E_2].$$

Definition 8: If $\text{Adv}_{\mathcal{A}}^{\text{DOC-EITA}}(\kappa)$ is negligible in κ for any PPT adversary, the VC-KASE scheme is DOC-EITA secure.

V. PROPOSED VC-KASE SCHEME

In this section, we describe our VC-KASE scheme and show its correctness.

A. Construction

- 1) *Setup*($1^\kappa, n$): Given a security parameter κ and the number of documents n owned by the data owner, the data owner initializes the system as follows.
 - a) Generate a symmetric bilinear map group $\mathbb{PG} = (p, \mathcal{G}, \mathcal{G}_T, e(\cdot, \cdot))$, where p is the order of $\mathcal{G}$ and $\mathcal{G}_T$, and $2^\kappa \leq p \leq 2^{\kappa+1}$.
 - b) Randomly select a generator $g \in \mathcal{G}$ and $\alpha \in \mathbb{Z}_p^*$ and calculate $g_i = g^{(\alpha^i)}$ for $i = 1, 2, \dots, n, n+2, \dots, 2n$.
 - c) Specify hash functions $H: \{0, 1\}^* \rightarrow \mathbb{Z}_p^*$ and $H_1: \{0, 1\}^* \rightarrow \mathcal{G}^*$.

Finally, the data owner publishes the system parameters

$$\text{params} = \{\mathbb{PG}, (g, g_1, \dots, g_n, g_{n+2}, \dots, g_{2n}), H, H_1\}.$$

- 2) *KeyGen_o*: The data owner randomly picks $\beta, \lambda \in \mathbb{Z}_p^*$ and computes $v = g^\beta$, and outputs the public-private key pair (pk_o, sk_o)

$$pk_o = v, sk_o = (\beta, \lambda).$$

- 3) *KeyGen_s*: The cloud server randomly picks $\gamma \in \mathbb{Z}_p^*$ and computes $u = g^\gamma$, and outputs the public-private key pair (pk_s, sk_s)

$$(pk_s, sk_s) = (u, \gamma).$$

- 4) *Encrypt*($pk_o, sk_o, pk_s, i, W_i, cf_i$): Input document identifier i , keyword set W_i and document ciphertext cf_i , the data owner randomly picks $r, t \in \mathbb{Z}_p^*$ and computes the keyword ciphertext and signature as follows.

- a) Generate the keyword ciphertext CT_i

$$\begin{aligned} c_1 &= g^r, c_2 = g^{\lambda r}, c_3 = g^{\lambda t} \\ c_4 &= (v \cdot g_i)^{\lambda r}, c_5 = e(g_1, g_n)^{\lambda r} \\ CW_i &= \{c_{i,j}\}_{j=1}^m = \left\{ pk_s^{H(w_{i,j})\beta r + t} \right\}_{j=1}^m. \end{aligned}$$

- b) Generate the signature σ_i

$$\sigma_i = H_1(i || cf_i)^\beta.$$

Finally, the data owner outputs $\{CT_i, cf_i, \sigma_i\}$ associated with the document identifier i , where $CT_i = \{c_1, c_2, c_3, c_4, c_5, CW_i\}$.

- 5) *Extract*(sk_o, S): Input the document subset $S \subseteq \{1, \dots, n\}$, the data owner computes the aggregate key

$$K_1^S = \prod_{j \in S} g_{n+1-j}^\beta, K_2^S = g^\lambda.$$

Finally, the data owner sends $K_{agg}^S = \{K_1^S, K_2^S\}$ and S to the user via a secure channel, thus authorizing the user to access documents in the set S .

- 6) *Trapdoor*(pk_o, K_{agg}^S, Q): Input the queried keyword set $Q = \{w'_{j_1}, w'_{j_2}, \dots, w'_{j_l}\}$, the user randomly picks $x, y \in \mathbb{Z}_p^*$ and computes the trapdoor Tr for all documents related to K_{agg}^S as follows:

$$T_1 = K_1^S \cdot v^{\sum_{k=1}^l H(w'_{j_k})x} \cdot g^y, \\ T_2 = g^x, T_3 = (K_2^S)^x, T_4 = (K_2^S)^y, T_5 = \{j_k\}_{k=1}^l.$$

Finally, the user submits $Tr = \{T_1, T_2, T_3, T_4, T_5\}$ and S to the cloud server, where T_5 is the keyword field set of the queried keyword set.

- 7) *Test*($Tr, S, i, CT_i, sk_s, cf_i, \sigma_i$): The cloud server first initializes two sets Σ and C . Input the trapdoor Tr and document set S , the cloud server performs the search as follows.

- a) Adjust Tr to the actual trapdoor Tr_i for the document with identifier $i \in S$

$$T_{i,1} = T_1 \cdot \prod_{j \in S, j \neq i} g_{n+1-j+i} \\ T_{i,2} = T_2, T_{i,3} = T_3, T_{i,4} = T_4, T_{i,5} = T_5.$$

- b) Input the keyword ciphertext CT_i to locate the corresponding keyword ciphertext set $\{c_{i,j_k}\}_{k=1}^l$ in CT_i based on the queried keyword field set $T_{i,5}$. Verify whether (1) holds

$$\frac{e(T_{i,1}, c_2)}{e\left(\prod_{j \in S} g_{n+1-j}, c_4\right) e(c_1, T_{i,4})} \\ \stackrel{?}{=} \frac{e\left(\left(\prod_{k=1}^l c_{i,j_k}\right)^{\frac{1}{sk_s}}, T_{i,3}\right)}{c_5 \cdot e(c_3, T_{i,2})^l} \quad (1)$$

If (1) holds, the cloud server returns 1, then adds σ_i to Σ and $\{i, cf_i\}$ to C , otherwise returns 0.

Finally, the cloud server computes the aggregate signature $\sigma = \prod_{i \in S} \sigma_i$ based on Σ for all search documents related to Tr , and then returns σ and C to the user.

- 8) *Verify*(pk_o, σ, C): Input the aggregate signature σ and document identifier-ciphertext set C , the user verifies whether (2) holds

$$e(g, \sigma) \stackrel{?}{=} e\left(v, \prod_{i \in S} H_1(i||cf_i)\right) \quad (2)$$

If (2) holds, the user returns 1, otherwise returns 0.

B. Correctness

In this section, we show the correctness of *Test* algorithm and *Verify* algorithm as follows.

- 1) *Test*: First, we calculate the left side of (1)

$$\frac{e(T_{i,1}, c_2)}{e\left(\prod_{j \in S} g_{n+1-j}, c_4\right) e(c_1, T_{i,4})} \\ = \frac{e\left(K_1^S \cdot v^{\sum_{k=1}^l H(w'_{j_k})x} \cdot g^y \cdot \prod_{j \in S, j \neq i} g_{n+1-j+i}, g^{\lambda r}\right)}{e\left(\prod_{j \in S} g_{n+1-j}, (v \cdot g_i)^{\lambda r}\right) e(g^r, (K_2^S)^y)} \\ = \frac{e\left(\prod_{j \in S} g_{n+1-j}^\beta \cdot g^{\lambda r}\right) e\left(\prod_{j \in S, j \neq i} g_{n+1-j+i}, g^{\lambda r}\right)}{e\left(\prod_{j \in S} g_{n+1-j}, g^{\beta \lambda r}\right) e\left(\prod_{j \in S} g_{n+1-j}, g_i^{\lambda r}\right)} \\ = \frac{e\left(g^{\sum_{k=1}^l H(w'_{j_k})\beta x + y}, g^{\lambda r}\right)}{e(g^r, g^{\lambda y})} \\ = \frac{e(g, g)^{\sum_{k=1}^l H(w'_{j_k})\beta x \lambda r}}{e(g_1, g_n)^{\lambda r}}.$$

Then, we have the right-hand side

$$\frac{e\left(\left(\prod_{k=1}^l c_{i,j_k}\right)^{\frac{1}{sk_s}}, T_{i,3}\right)}{c_5 \cdot e(c_3, T_{i,2})} \\ = \frac{e\left(g^{\sum_{k=1}^l H(w_{i,j_k})\beta r + lt}, g^{\lambda x}\right)}{e(g_1, g_n)^{\lambda r} \cdot e(g^{\lambda t}, g^x)^l} \\ = \frac{e(g, g)^{\sum_{k=1}^l H(w_{i,j_k})\beta r \lambda x}}{e(g_1, g_n)^{\lambda r}}.$$

Therefore, if the aggregate key is valid and the keywords of the document match the queried keywords in the corresponding fields, then $(w_{i,j_1} = w'_{j_1}) \wedge (w_{i,j_2} = w'_{j_2}) \wedge \dots \wedge (w_{i,j_l} = w'_{j_l})$ holds, (1) holds.

- 2) *Verify*. We calculate

$$e(g, \sigma) \\ = e\left(g, \prod_{i \in S} \sigma_i\right) \\ = e\left(g, \prod_{i \in S} H_1(i||cf_i)^\beta\right) \\ = e\left(v, \prod_{i \in S} H_1(i||cf_i)\right).$$

Clearly, if C is a valid document identifier-ciphertext set and σ is the aggregate signature corresponding to C , then (2) holds.

VI. SECURITY ANALYSIS

We have conducted formal security proofs under four security models: CT-IND-CKA, TR-IND-CKA, EU-AK-CKA, and DOC-EITA, which respectively guarantee ciphertext privacy,

trapdoor privacy, aggregate key unforgeability, and aggregate key and keyword privacy against EITA.

Lemma 1: The VC-KASE scheme is CT-IND-CKA secure assuming the hardness of the divisible CDH problem and the CDH problem.

Proof: Suppose there is a PPT adversary $\mathcal{A}_1$ that can win the CT-IND-CKA security game with advantage $\varepsilon = \text{non} - \text{negl}(\kappa)$, we can build a PPT algorithm $\mathcal{B}$ to solve the divisible CDH problem. Given a divisible CDH instance (g, g^a, g^b) in the group $\mathcal{G}$. $\mathcal{B}$ works as follows.

Game CT-1: Suppose $\mathcal{A}_1$ is a cloud server.

Setup: $\mathcal{A}_1$ submits set $S^* \subseteq [1, n]$ to challenge. $\mathcal{B}$ randomly picks $i^* \in S^*$ and sends it to $\mathcal{A}_1$. $\mathcal{B}$ randomly select a generator $g \in \mathcal{G}$ and $\alpha \in Z_p^*$ and calculate $g_i = g^{(\alpha^i)}$ for $i = 1, 2, \dots, n, n+2, \dots, 2n$, specify two hash functions $H: \{0, 1\}^* \rightarrow Z_p^*$ and $H_1: \{0, 1\}^* \rightarrow \mathcal{G}^*$. Therefore, $\text{params} = \{\mathbb{P}\mathbb{G}, (g, g_1, \dots, g_n, g_{n+2}, \dots, g_{2n}), H, H_1\}$. It then randomly picks $\beta, \lambda, \gamma \in Z_p^*$ and sets $pk_o = v = g^\beta, sk_o = (\beta, \lambda), pk_s = u = g^\gamma, sk_s = \gamma$. $\mathcal{B}$ sends $\text{params}, pk_o, (pk_s, sk_s)$ to $\mathcal{A}_1$.

Phase 1: $\mathcal{A}_1$ issues the following queries adaptively.

1) **Ciphertext Query**(i, W): Given a document identifier i and the related keyword set W , $\mathcal{B}$ randomly picks $r, t \in Z_p^*$ and computes the keyword ciphertext

$$CT_i = (c_1, c_2, c_3, c_4, c_5, CW_i) \\ = (g^r, g^{\lambda r}, g^{\lambda t}, (v \cdot g_i)^{\lambda r}, e(g_1, g_n)^{\lambda r}, \{u^{H(w_i)\beta r + t}\}_{w_i \in W}).$$

Then, $\mathcal{B}$ returns it to $\mathcal{A}_1$.

2) **Trapdoor query**(Q): Given a queried keyword set $Q = \{w'_{j_1}, w'_{j_2}, \dots, w'_{j_l}\}$, $\mathcal{B}$ randomly picks $x, y \in Z_p^*$ and computes the trapdoor ciphertext related to $K_{\text{agg}}^{S^*}$

$$Tr = (T_1, T_2, T_3, T_4, T_5) \\ = \left(\prod_{j \in S^*} g_{n+1-j}^\beta \cdot g^{\sum_{k=1}^l H(w'_{j_k})\beta x + y}, g^x, g^{\lambda x}, g^{\lambda y}, \{j_k\}_{k=1}^l \right).$$

Then, $\mathcal{B}$ returns it to $\mathcal{A}_1$.

Challenge: $\mathcal{A}_1$ submits two distinct challenge keyword sets $W_0^*, W_1^* (|W_0^*| = |W_1^*|)$ to $\mathcal{B}$. Then $\mathcal{B}$ picks a random bit $c \in \{0, 1\}$ and randomly picks $t \in Z_p^*$. It then sets $b = \lambda a$ and computes the challenge ciphertext associated with the document identifier i^*

$$CT^* = (c_1^*, c_2^*, c_3^*, c_4^*, c_5^*, CW^*) \\ = \left(g^a, g^b, g^{\lambda t}, (v \cdot g_{i^*})^b, e(g_1, g_n)^b, \{u^{H(w_{i^*})\beta a + t}\}_{w_{i^*} \in W_c^*} \right).$$

Then, $\mathcal{B}$ returns it to $\mathcal{A}_1$. For document identifier i^* , W_0^* and W_1^* cannot have been queried. Moreover, W_0^* and W_1^* are not equal to the queried Q .

For $w_{i^*} \in W_c^*$, $\mathcal{A}_1$ needs to solve the divisible CDH problem to get $g^\lambda = g^{\frac{b}{a}}$ and then can computes

$$e\left(\left(u^{H(w_{i^*})\beta a + t}\right)^{\frac{1}{sk_s}}, g^\lambda\right) = e\left((g^\beta)^{H(w_{i^*})}, g^b\right) \cdot e(g^{\lambda t}, g).$$

Phase 2: $\mathcal{A}_1$ issues ciphertext queries and trapdoor queries as in Phase 1. The limit is that $(i, W) \neq (i^*, W_0^*), (i, W) \neq (i^*, W_1^*)$ for ciphertext query(i, W) and $Q \neq W_0^*, Q \neq W_1^*$ for trapdoor query(Q).

Guess: $\mathcal{A}_1$ returns $c' \in \{0, 1\}$ and wins if $c' = c$.

The above simulation and the actual attack are indistinguishable. Thus, if $\mathcal{A}_1$ can win the CT-IND-CKA security game with nonnegligible advantage

$$\text{Adv}_{\mathcal{A}_1}^{\text{CT-IND-CKA}}(\kappa) = \left| \Pr[c' = c] - \frac{1}{2} \right| = \varepsilon,$$

$\mathcal{B}$ can solve the divisible CDH problem with advantage ε . ■

Proof: The proof is similar to that of Game CT-1. Given a CDH instance (g, g^a, g^b) in the group $\mathcal{G}$. $\mathcal{B}$ works as follows.

Game CT-2: Suppose $\mathcal{A}_2$ is a user.

Setup: $\mathcal{A}_2$ submits a document identifier $i^* (i^* \in [1, n])$ to challenge. $\mathcal{B}$ generates params in the same way as in Game CT-1. It then randomly picks $a, \lambda, b \in Z_p^*$ and sets $pk_o = v = g^a, sk_o = (a, \lambda), pk_s = u = g^b, sk_s = b$. $\mathcal{B}$ sends $\text{params}, pk_o, pk_s$ to $\mathcal{A}_2$.

Phase 1: $\mathcal{A}_2$ issues the following queries adaptively.

Aggregate Key query(S): Given a document identifier set $S (i^* \notin S)$, $\mathcal{B}$ computes

$$K_1^S = \prod_{j \in S} g_{n+1-j}^a, K_2^S = g^\lambda.$$

Then, $\mathcal{B}$ returns $K_{\text{agg}}^S = (K_1^S, K_2^S)$ to $\mathcal{A}_2$.

Ciphertext query as in Game CT-1.

Challenge: $\mathcal{A}_2$ submits two distinct challenge keyword sets $W_0^*, W_1^* (|W_0^*| = |W_1^*|)$ to $\mathcal{B}$. Then $\mathcal{B}$ picks a random bit $c \in \{0, 1\}$ and randomly picks $r, t \in Z_p^*$, computes the challenge ciphertext associated with the document identifier i^*

$$CT^* = (c_1^*, c_2^*, c_3^*, c_4^*, c_5^*, CW^*) = \\ (g^r, g^{\lambda r}, g^{\lambda t}, (v \cdot g_{i^*})^{\lambda r}, e(g_1, g_n)^{\lambda r}, \{g^{H(w_{i^*})abr + bt}\}_{w_{i^*} \in W_c^*}).$$

Then, $\mathcal{B}$ returns it to $\mathcal{A}_2$. For document identifier i^* , W_0^* and W_1^* cannot have been queried.

For $w_{i^*} \in W_c^*$, $\mathcal{A}_2$ computes

$$\frac{e(g^{H(w_{i^*})abr + bt}, g^\lambda)}{e(g^b, g^{\lambda t})} = e((g^{\lambda r})^{H(w_{i^*})}, g^{ab}).$$

$\mathcal{A}_2$ needs to solve the CDH problem to get g^{ab} .

Phase 2: $\mathcal{A}_2$ issues aggregate key queries and ciphertext queries as in Phase 1. The limit is that $(i, W) \neq (i^*, W_0^*)$ and $(i, W) \neq (i^*, W_1^*)$ for ciphertext query(i, W).

Guess: $\mathcal{A}_2$ returns $c' \in \{0, 1\}$ and wins if $c' = c$.

The above simulation and the actual attack are indistinguishable. Thus, if $\mathcal{A}_2$ can win the CT-IND-CKA security game with nonnegligible advantage

$$\text{Adv}_{\mathcal{A}_2}^{\text{CT-IND-CKA}}(\kappa) = \left| \Pr[c' = c] - \frac{1}{2} \right| = \varepsilon,$$

$\mathcal{B}$ can solve the divisible CDH problem with advantage ε . ■

Lemma 2: The VC-KASE scheme is TR-IND-CKA secure assuming the hardness of the CDH problem. Moreover, by embedding the data owner's private keys within the encrypted data, it can prevent the attacker from generating valid ciphertext for matching to resist offline KGA.

Proof: The proof is similar to Game CT-1. Given a CDH instance (g, g^a, g^b) in the group $\mathcal{G}$. $\mathcal{B}$ works as follows.

Game TR-1: Suppose $\mathcal{A}_1$ is a cloud server.

Setup: $\mathcal{A}_1$ submits set $S^* \subseteq [1, n]$ to challenge. $\mathcal{B}$ randomly picks $i^* \in S^*$ and sends it to $\mathcal{A}_1$. $\mathcal{B}$ generates $params$ in the same way as in Game CT-1. It then randomly picks $a, \lambda, \gamma \in \mathbb{Z}_p^*$ and sets $pk_o = v = g^a, sk_o = (a, \lambda), pk_s = u = g^\gamma, sk_s = \gamma$. $\mathcal{B}$ sends $params, pk_o, (pk_s, sk_s)$ to $\mathcal{A}_1$.

Phase 1: $\mathcal{A}_1$ issues the following queries adaptively.

Ciphertext query and trapdoor query as in Game CT-1.

Challenge: $\mathcal{A}_1$ submits two distinct challenge keyword sets $Q_0^*, Q_1^* (|Q_0^*| = |Q_1^*|)$ in the same keyword fields to $\mathcal{B}$. Then $\mathcal{B}$ picks a random bit $c \in \{0, 1\}$ and randomly picks $y \in \mathbb{Z}_p^*$, computes the challenge trapdoor associated with the document identifier i^*

$$Tr^* = (T_1^*, T_2^*, T_3^*, T_4^*, T_5^*) = \left(\prod_{j \in S^*} g_{n+1-j}^a \cdot g^{\sum_{k=1}^l H(w_{jk}^*)ab+y}, g^b, g^{\lambda b}, g^{\lambda y}, \{j_k\}_{k=1}^l \right).$$

Then, $\mathcal{B}$ returns it to $\mathcal{A}_1$. Q_0^* and Q_1^* cannot have been queried. Moreover, Q_0^* and Q_1^* is not equal to the queried W associated with the document identifier i^* .

For Q_c^* , $\mathcal{A}_1$ computes

$$\begin{aligned} & \frac{e\left(\prod_{j \in S^*} g_{n+1-j}^a \cdot g^{\sum_{k=1}^l H(w_{jk}^*)ab+y} \cdot \prod_{j \in S, j \neq i^*} g_{n+1-j+i}, g^{\lambda r}\right)}{e\left(\prod_{j \in S} g_{n+1-j}, (v \cdot g_i)^{\lambda r}\right) e(g^r, g^{\lambda y}) (e(g_1, g_n)^{\lambda r})^{-1}} \\ &= \frac{e\left(\prod_{j \in S} g_{n+1-j}^a, g^{\lambda r}\right) e\left(\prod_{j \in S, j \neq i^*} g_{n+1-j+i}, g^{\lambda r}\right)}{e\left(\prod_{j \in S} g_{n+1-j}, g^{a\lambda r}\right) e\left(\prod_{j \in S} g_{n+1-j}, g_i^{\lambda r}\right) e(g_1, g_n)^{-\lambda r}} \\ & \quad \cdot \frac{e\left(g^{\sum_{k=1}^l H(w_{jk}^*)ab+y}, g^{\lambda r}\right)}{e(g^r, g^{\lambda y})} \\ &= e\left(g^{\sum_{k=1}^l H(w_{jk}^*)ab}, g^{\lambda r}\right) \\ &= e\left(g^{ab}, (g^{\lambda r})^{\sum_{k=1}^l H(w_{jk}^*)}\right). \end{aligned}$$

$\mathcal{A}_1$ needs to solve the CDH problem to get g^{ab} .

Phase 2: $\mathcal{A}_1$ issues ciphertext queries and trapdoor queries as in Phase 1. The limit is that $(i, W) \neq (i^*, Q_0^*), (i, W) \neq (i^*, Q_1^*)$ for ciphertext query (i, W) and $Q \neq Q_0^*, Q \neq Q_1^*$ for trapdoor query (Q) .

Guess: $\mathcal{A}_1$ returns $c' \in \{0, 1\}$ and wins if $c' = c$.

The above simulation and the actual attack are indistinguishable. Thus, if $\mathcal{A}_1$ can win the TR-IND-CKA security game with nonnegligible advantage

$$\text{Adv}_{\mathcal{A}_1}^{\text{TR-IND-CKA}}(\kappa) = \left| \Pr[c' = c] - \frac{1}{2} \right| = \varepsilon,$$

$\mathcal{B}$ can solve the divisible CDH problem with advantage ε . ■

Proof: The proof is similar to Game TR-1 and we omit Game TR-2 for compactness. ■

Lemma 3: The VC-KASE scheme is EU-AK-CKA secure assuming the hardness of the n-DHE problem.

Proof: The proof is similar to that of Game CT-1. Let $g_i = g^{(\alpha^i)}$ for a generator $g \in \mathcal{G}$ and $\alpha \in \mathbb{Z}_p^*$. Given a n-DHE instance $(g, g_1, g_2, \dots, g_n, g_{n+2}, \dots, g_{2n})$ in the group $\mathcal{G}$. $\mathcal{B}$ works as follows.

Setup: $\mathcal{A}$ submits a document identifier $i^* (i^* \in [1, n])$ to challenge. $\mathcal{B}$ generates $params$ in the same way as in Game CT-1. It then randomly picks $\beta, \lambda, \gamma \in \mathbb{Z}_p^*$ and sets $pk_o = v = g^\beta, sk_o = (\beta, \lambda), pk_s = u = g^\gamma, sk_s = \gamma$. $\mathcal{B}$ sends $params, pk_o, pk_s$ to $\mathcal{A}$.

Phase 1: $\mathcal{A}$ issues the following queries adaptively.

Aggregate Key Query(S): Given a document identifier set $S (i^* \notin S)$, $\mathcal{B}$ computes

$$K_1^S = \prod_{j \in S} g_{n+1-j}^\beta, K_2^S = g^\lambda.$$

Then, $\mathcal{B}$ returns $K_{\text{agg}}^S = (K_1^S, K_2^S)$ to $\mathcal{A}$.

Ciphertext query as in Game CT-1.

Forge: $\mathcal{A}$ returns a forged aggregate key $K_{\text{agg}}^{S^*}$ of document identifier set $S^* (i^* \in S^*)$. $\mathcal{B}$ sets $g^\beta = g^{\beta' - \alpha^{i^*}} = g^{\beta'} g_i^{*-1}$.

If $K_{\text{agg}}^{S^*}$ is valid, namely

$$K_1^{S^*} = \prod_{j \in S^*} g_{n+1-j}^{\beta' - \alpha^{i^*}} = g_{n+1-i^*}^{\beta' - \alpha^{i^*}} \cdot \prod_{j \in S^*, j \neq i^*} g_{n+1-j}^{\beta' - \alpha^{i^*}}, K_2^{S^*} = g^\lambda.$$

Then, $\mathcal{B}$ can compute

$$\begin{aligned} & \frac{g_{n+1-i^*}^{\beta'} \cdot \prod_{j \in S^*, j \neq i^*} g_{n+1-j}^{\beta'} \cdot \prod_{j \in S^*, j \neq i^*} g_{n+1-j+i^*}^{-1}}{K_1^{S^*}} \\ &= \frac{g_{n+1-i^*}^{\beta'} \cdot \prod_{j \in S^*, j \neq i^*} g_{n+1-j}^{\beta'} \cdot \prod_{j \in S^*, j \neq i^*} g_{n+1-j+i^*}^{-1}}{g_{n+1-i^*}^{\beta'} \cdot (g_{n+1-i^*}^{\alpha^{i^*}})^{-1} \cdot \prod_{j \in S^*, j \neq i^*} g_{n+1-j}^{\beta' - \alpha^{i^*}}} \\ &= \frac{\prod_{j \in S^*, j \neq i^*} g_{n+1-j+i^*}^{-1}}{(g_{n+1-i^*}^{\alpha^{i^*}})^{-1} \cdot \prod_{j \in S^*, j \neq i^*} (g_{n+1-j}^{\alpha^{i^*}})^{-1}} \\ &= g^{(\alpha^{n+1})}. \end{aligned}$$

The above simulation and the actual attack are indistinguishable. Thus, if $\mathcal{A}$ returns a valid forged aggregate key $K_{\text{agg}}^{S^*}$, $\mathcal{B}$ can use the forged key to solve the n-DHE problem. Under the n-DHE assumption, the probability that $\mathcal{A}$ returns a valid forged aggregate key is ϵ , then $\mathcal{B}$ can solve the n-DHE problem with advantage ϵ . ■

Lemma 4: The VC-KASE scheme is DOC-EITA secure assuming the hardness of the DL problem and the divisible CDH problem.

Proof: Suppose there is a PPT adversary $\mathcal{A}$ that can win the DOC-EITA security game with advantage $\varepsilon = \text{non-negl}(\kappa)$, we can build a PPT algorithm $\mathcal{B}$ to solve the divisible CDH problem. Given a DL instance (g, g^c) and a divisible CDH instance (g, g^a, g^b) in the group $\mathcal{G}$. $\mathcal{B}$ works as follows.

Setup: $\mathcal{A}$ submits a document identifier $i^* (i^* \in [1, n])$ to challenge. $\mathcal{B}$ generates $params$ in the same way as in Game CT-1. It then randomly picks $\beta, a, \gamma \in \mathbb{Z}_p^*$ and sets $pk_o = v = g^\beta, sk_o = (\beta, a), pk_s = u = g^\gamma, sk_s = \gamma$. $\mathcal{B}$ sends $params, pk_o, pk_s$ to $\mathcal{A}$.

Phase 1: $\mathcal{A}$ issues the following queries adaptively.

- 1) *Aggregate Key Query*(S): Given a document identifier set $S(i^* \notin S)$, $\mathcal{B}$ computes

$$K_1^S = \prod_{j \in S} g_{n+1-j}^\beta, K_2^S = g^a.$$

Then, $\mathcal{B}$ returns $K_{\text{agg}}^S = (K_1^S, K_2^S)$ to $\mathcal{A}$.

- 2) *Trapdoor query*(S^*): Given a document identifier set $S^*(i^* \in S^*)$, $\mathcal{B}$ randomly selects a queried keyword set $Q = \{w_{j_1}, w_{j_2}, \dots, w_{j_l}\}$ and picks $c \in Z_p^*$. It then sets $y = \frac{b}{a}$ and computes the trapdoor ciphertext related to $K_{\text{agg}}^{S^*}$

$$\begin{aligned} Tr &= (T_1, T_2, T_3, T_4, T_5) \\ &= \left(K_1^{S^*} \cdot v^{\sum_{k=1}^l H(w_{j_k})c} \cdot g^y, g^c, g^{ac}, g^b, \{j_k\}_{k=1}^l \right). \end{aligned}$$

Then, $\mathcal{B}$ returns it to $\mathcal{A}$. The trapdoor Tr corresponds to a result set $S'(i^* \notin S')$.

Guess: The adversary $\mathcal{A}$ is allowed a limited number of attempts and wins if either of the following conditions is satisfied. Prior to launching a specific attack, $\mathcal{A}$ first infers the keyword set embedded in a previously queried trapdoor Tr , which serves as a prerequisite for both attack strategies described below.

For a queried trapdoor Tr corresponding to a result set S' with $i^* \notin S'$, $\mathcal{A}$ selects a document identifier $i \in S'$ and leverages a previously queried aggregate key K_{agg}^S granting access to $i \in S$. By correlating the access rights granted by K_{agg}^S to i with the trapdoor Tr , $\mathcal{A}$ can derive the keyword set Q embedded in Tr .

Having inferred the keyword set $Q = \{w_{j_1}, w_{j_2}, \dots, w_{j_l}\}$ embedded in Tr , $\mathcal{A}$ can subsequently launch one of the following attacks.

- 1) $\mathcal{A}$ aims to infer a keyword w^* contained in the challenge document i^* . To mount the attack, $\mathcal{A}$ forges a valid trapdoor Tr^* for the keyword w^* and the document identifier i^*

$$\begin{aligned} Tr^* &= (T_1^*, T_2, T_3, T_4, T_5^*) \\ &= \left(K_1^{S^*} \cdot v^{H(w^*)c} \cdot g^y, g^c, g^{ac}, g^b, \{j_k\}_{k=1}^l \right). \end{aligned}$$

where T_2, T_3, T_4 are reused and $T_5^* = \{j_k\}_{k=1}^l$ contains the index of w^* . To compute T_1^* , and computes

$$T_1^* = \frac{T_1}{\left(v^{\sum_{k=1}^l H(w_{j_k})} \right)^c} \cdot \left(v^{H(w^*)} \right)^c.$$

$\mathcal{A}$ needs to solve the DL problem to get c from g^c .

After submitting Tr^* to the cloud server, if the returned result set S^* contains i^* , $\mathcal{A}$ infers that w^* is associated with i^* . Since the keyword space is finite, $\mathcal{A}$ can iterate over all candidates and repeat this process to eventually determine one or more keywords linked to i^* .

- 2) $\mathcal{A}$ aims to recover the aggregate key $K_{\text{agg}}^{S^*} = (K_1^{S^*}, K_2^{S^*})$ for a document set $S^*(i^* \in S^*)$, where $K_{\text{agg}}^{S^*}$ is inferred from the previously queried trapdoor $Tr = (T_1, T_2, T_3, T_4, T_5)$. Since the component $K_2^{S^*} = g^a$

remains the same across aggregate keys generated by the same authority, $\mathcal{A}$ focuses on extracting $K_1^{S^*}$ from the trapdoor component $T_1 = K_1^{S^*} \cdot v^{\sum_{k=1}^l H(w_{j_k})c} \cdot g^y$, $\mathcal{A}$ computes

$$K_1^{S^*} = \frac{T_1}{\left(v^{H(w_{j_k})} \right)^c \cdot g^y}.$$

$\mathcal{A}$ needs to solve both the DL problem to get c from g^c and the divisible CDH problem to get g^y from g^a and $g^b = g^{ay}$.

The above simulation and the actual attack are indistinguishable. Let E_1 denote the event that $\mathcal{A}$ correctly outputs such a w^* , and E_2 denote the event that it successfully outputs such a $K_{\text{agg}}^{S^*}$. Thus, if $\mathcal{A}$ can win the DOC-EITA security game with nonnegligible advantage

$$\text{Adv}_{\mathcal{A}}^{\text{DOC-EITA}}(\kappa) = \Pr[E_1 \vee E_2] = \varepsilon.$$

$\mathcal{B}$ can solve the DL problem or solve both the DL problem and the divisible CDH problem with advantage ϵ . ■

VII. PERFORMANCE

A. Theoretical Evaluation

For the theoretical evaluation, we primarily compare the computational and communication costs with the schemes of [9], [10], and [34]. Since the authentication mechanism of [34] is universal, we only focus on its verification without considering its authentication. To ensure consistency, we integrate trapdoor adjustment and search matching into the *Test* algorithm.

First, we show the computational costs of major algorithms in Table III. In *Setup*, *KeyGen_o*, *KeyGen_s* and *Extract* algorithms, the computational costs across all schemes are similar due to analogous operations, except that [34] does not set the server key. In *Encrypt* algorithm, the total computational cost of the proposed VC-KASE is slightly higher than [9] for signature generation and enhanced security, yet it is lower than [10] and [34]. In *Trapdoor* and *Test* algorithms, the computation costs of [10] and [34] scale linearly with the number of keyword fields, while our scheme and [9] maintain a relatively constant cost. Specifically, for multikeyword queries, [10] and [34] need to generate separate trapdoors and test each keyword, whereas our scheme and [9] only need to compute a single aggregate trapdoor for multiple keywords of interest and test it. Additionally, our *verify* algorithm is more efficient than [34], which requires multiple verifications for multikeyword queries.

Then, we show the communication costs in Table IV. The ciphertext and aggregate key sizes of these schemes differ slightly, with both our scheme and [34], including signatures in the ciphertext. Our scheme features a larger trapdoor size than that of [9] by two auxiliary values to enhance security. While [10] and [34] generate a separate trapdoor for each keyword, resulting in a linear relationship between the size of the total trapdoor generated and the number of keywords of interest. Additionally, our scheme performs verification only

TABLE III
COMPUTATION COMPARISON

	vaKASE [34]	dKASE [10]	KASE-CQ [9]	VC-KASE
Setup	$(2n-1)E_G + (2n-2)M_{Z_p}$	$(2n-1)E_G + (2n-2)M_{Z_p}$	$(2n-1)E_G + (2n-2)M_{Z_p}$	$(2n-1)E_G + (2n-2)M_{Z_p}$
KeyGen_o	$2E_G$	$2E_G$	$2E_G$	E_G
KeyGen_s	-	E_G	E_G	E_G
Encrypt	$2mP + 4E_G + 2mE_{G_T} + 2M_G + mM_{G_T}$	$(2m+1)P + 2E_G + (m+1)E_{G_T} + M_G + mM_{G_T}$	$2P + (m+3)E_G + 2E_{G_T} + M_G + (m+2)M_{Z_p}$	$P + (m+5)E_G + E_{G_T} + M_G + (2m+2)M_{Z_p}$
Extract	$E_G + (S -1)M_G$	$E_G + (S -1)M_G$	$E_G + (S -1)M_G$	$2E_G + (S -1)M_G$
Trapdoor	lM_G	$l(2E_G + 2M_G)$	$3E_G + 2M_G + (l-1)M_{Z_p}$	$5E_G + 2M_G + lM_{Z_p}$
Test	$l S (2P + (2 S -2)M_G + M_{G_T})$	$l S (3P + E_G + E_{G_T} + (2 S -1)M_G + 3M_{G_T})$	$ S (3P + E_G + E_{G_T} + (2 S +l-2)M_G + 3M_{G_T})$	$(d-1)M_G + S (5P + E_G + E_{G_T} + (2 S +l-3)M_G + 4M_{G_T})$
Verify	$l(2P + (2d+3)E_G + 2dM_G + (d+1)M_{Z_p})$	-	-	$2P + (d-1)M_G$

P : Bilinear pairing; M_{Z_p} : Multiplication in Z_p ; E_G : Exponentiation in G ; E_{G_T} : Exponentiation in G_T ; M_G : Multiplication in G ; M_{G_T} : Multiplication in G_T ; -: Not involved.

TABLE IV
COMMUNICATION COMPARISON

	vaKASE [34]	dKASE [10]	KASE-CQ [9]	VC-KASE
Ciphertext (Data Owner→Server)	$n(3 G + m G_T)$	$n(2 G + G_T + m \kappa)$	$n((m+3) G + 2 G_T)$	$n((m+5) G + G_T)$
Aggregate Key (Data Owner→User)	$ G $	$ G + Z_p $	$ G $	$2 G $
Trapdoor (User→Server)	$l G $	$2l G $	$2 G + l \kappa $	$4 G + l \kappa $
Verify	challenge (User→Server)	$ld Z_p $	-	-
	proof (Server→User)	$l(2 G + Z_p)$	-	$ G $

$|G|$: the bit-length of an element in G ; $|G_T|$: the bit-length of an element in G_T ; $|Z_p|$: the bit-length of an element in Z_p ; $|\kappa|$: the length of security parameter κ ; -: Not involved.

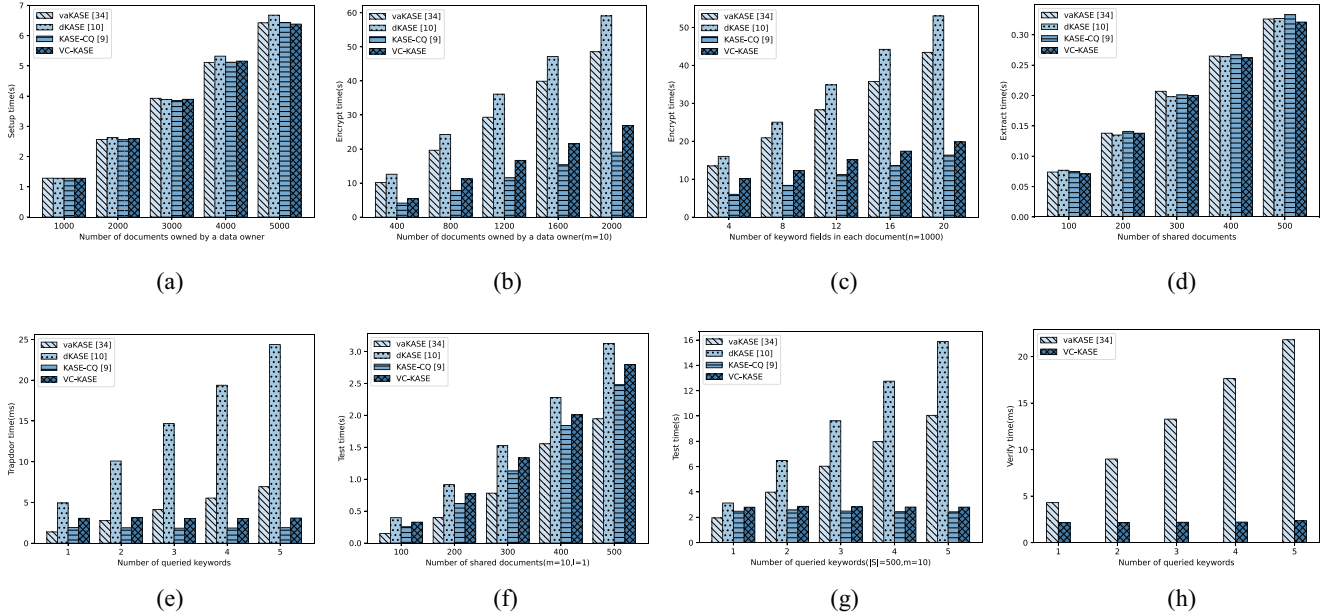


Fig. 3. Time cost comparison. (a) Setup time (documents). (b) Encryption time (documents). (c) Encryption time (keyword fields). (d) Extract time (shared documents). (e) Trapdoor time (queried keywords). (f) Test time (shared documents). (g) Test time (queried keywords). (h) Verify time (queried keywords).

using a single aggregate signature, whereas [34] requires the user to initiate the challenge first and necessitates multiple verifications for multikeyword queries.

B. Experimental Evaluation

To evaluate the performance of VC-KASE, we will calculate the system time cost compared to the schemes [9], [10], [34]. We conduct our experiments on Ubuntu 20.04.6 LTS operating

system, using GCC 9.4.0 programming language and PBC Library with the Type A elliptic curve parameters selected. The whole experiment is run on a Linux Server with 12th Gen Intel Core i9-12900H@ 2.90GHz (with 64bit OS, 1.9GB RAM). The security parameter is set to 512 bits.

Fig. 3(a) and (d) show that the *Setup* and *Extract* time costs of these schemes grow linearly with the number of documents owned by a data owner. Fig. 3(b) and (c) show the variation of the *Encrypt* time cost with the number of

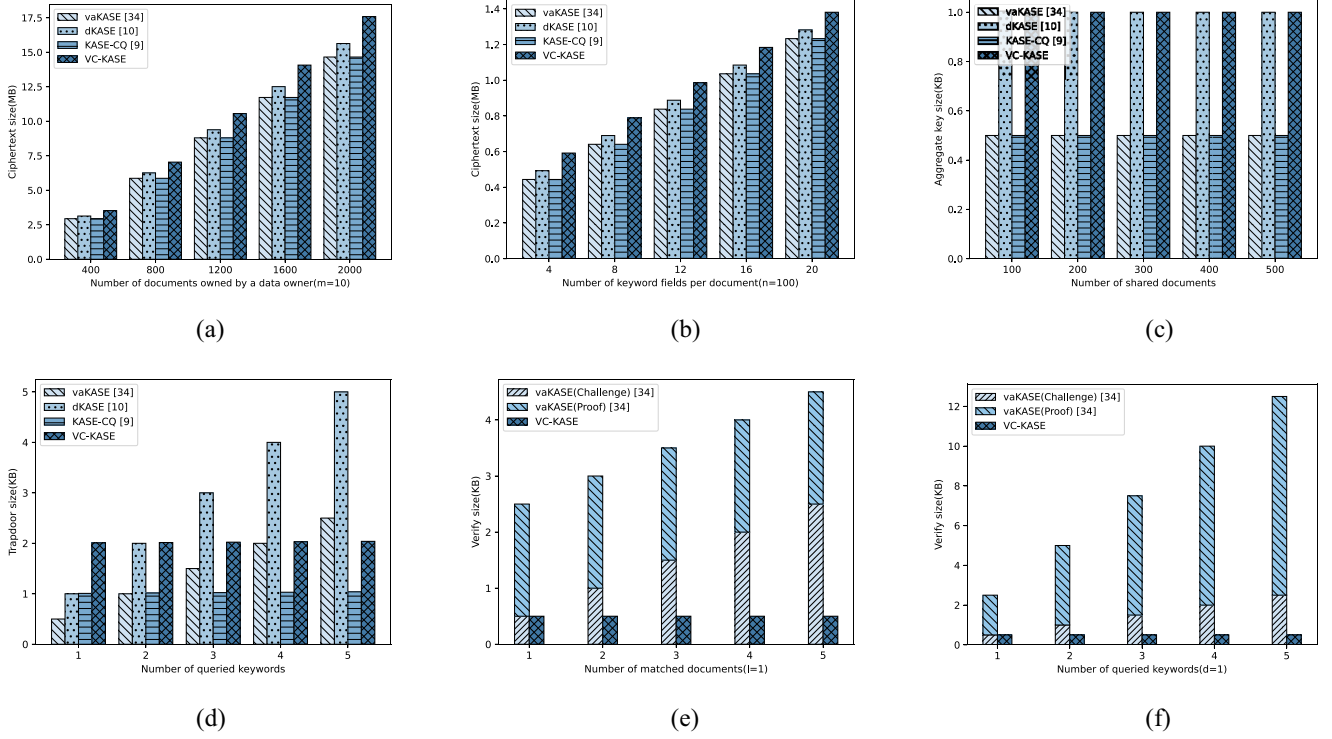


Fig. 4. Communication cost comparison. (a) Ciphertext size (documents). (b) Ciphertext size (keyword fields). (c) Aggregate key size (shared documents). (d) Trapdoor size (queried keywords). (e) Verify size (matched documents). (f) Verify size (queried keywords).

documents owned by a data owner and number of keyword fields in each document, respectively. The *Encrypt* time cost of all schemes is linearly related to these parameters, with our scheme having slightly higher time costs than [9] for signature generation and enhanced security, but lower than [10] and [34]. Fig. 3(e) and (g) shows that the *Trapdoor* and *Test* time cost for our scheme and [9] remain relative constant due to the support for conjunctive query, while the *Trapdoor* and *Test* time cost for [10] and [34] grows linearly with the number of queried keywords. To achieve trapdoor indistinguishability, our *Trapdoor* costs are slightly larger than [9]. Fig. 3(f) shows the *Test* time cost of all schemes grows linearly with the number of shared documents. We sacrifice a bit of efficiency for higher security, with a slightly higher time than [34] and [9]. Fig. 3(h) shows the time cost of *verify*, where we use aggregate signatures to effectively improve verification efficiency.

Fig. 4(a) and (b) show that ciphertext sizes grow linearly with the number of documents and keywords per document, with our scheme slightly larger due to embedded aggregate signatures. Fig. 4(c) shows that all schemes maintain constant aggregate key sizes as the number of shared documents increases, thanks to key aggregation, with ours slightly larger for enhanced security. Fig. 4(d) shows that trapdoor sizes in our scheme and [9] remain nearly constant due to support for conjunctive queries, while those in [10] and [34] grow linearly with the number of queried keywords. Fig. 4(e) and (f) show our scheme maintains constant verification cost using aggregate signatures, whereas [34]’s verification involves challenge and proof phases, with challenge cost increasing with both the number of matched documents and queried keywords, and proof cost growing with the number of queried keywords.

VIII. CONCLUSION

In this work, we present a novel attack named EITA, which captures a broader range of vulnerabilities in existing KASE schemes by targeting both the security of aggregate keys and the privacy of keywords in unauthorized documents. Furthermore, we propose a verifiable conjunctive field keyword SE scheme with aggregate keys, named VC-KASE. The proposed scheme supports both verification and conjunctive queries while resisting offline KGA and EITA. Security analysis and performance evaluation demonstrate that VC-KASE is secure and practical for IoT-enabled healthcare systems, effectively addressing privacy and verifiability concerns in these sensitive environments.

REFERENCES

- [1] R. Curtmola, J. Garay, S. Kamara, and R. Ostrovsky, “Searchable symmetric encryption: Improved definitions and efficient constructions,” in *Proc. 13th ACM Conf. Comput. Commun. Security*, 2006, pp. 79–88.
- [2] K. Ren, C. Wang, and Q. Wang, “Security challenges for the public cloud,” *IEEE Internet Comput.*, vol. 16, no. 1, pp. 69–73, Jan./Feb. 2012.
- [3] M. Li, S. Yu, Y. Zheng, K. Ren, and W. Lou, “Scalable and secure sharing of personal health records in cloud computing using attribute-based encryption,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 24, no. 1, pp. 131–143, Jan. 2013.
- [4] D. X. Song, D. Wagner, and A. Perrig, “Practical techniques for searches on encrypted data,” in *Proc. IEEE Symp. Security Privacy*, 2000, pp. 44–55.
- [5] D. Boneh, G. Di Crescenzo, R. Ostrovsky, and G. Persiano, “Public key encryption with keyword search,” in *Proc. Int. Conf. Theory Appl. Cryptograph. Techn.*, 2004, pp. 506–522.
- [6] B. Cui, Z. Liu, and L. Wang, “Key-aggregate searchable encryption (KASE) for group data sharing via cloud storage,” *IEEE Trans. Comput.*, vol. 65, no. 8, pp. 2374–2385, Aug. 2016.
- [7] J. W. Byun, H. S. Rhee, H.-A. Park, and D. H. Lee, “Off-line keyword guessing attacks on recent keyword search schemes over encrypted data,” in *Proc. Workshop Secure Data Manage.*, 2006, pp. 75–83.

- [8] R. Zhou, X. Zhang, X. Du, X. Wang, G. Yang, and M. Guizani, "File-centric multi-key aggregate keyword searchable encryption for Industrial Internet of Things," *IEEE Trans. Ind. Inform.*, vol. 14, no. 8, pp. 3648–3658, Aug. 2018.
- [9] J. Liu, B. Zhao, J. Qin, X. Hou, and J. Ma, "Key-aggregate searchable encryption supporting conjunctive queries for flexible data sharing in the cloud," *Inf. Sci.*, vol. 645, Oct. 2023, Art. no. 119336.
- [10] J. Liu, Z. Wei, J. Qin, B. Zhao, and J. Ma, "Verifiable key-aggregate searchable encryption with a designated server in multi-owner setting," *IEEE Trans. Services Comput.*, vol. 16, no. 6, pp. 4233–4247, Nov./Dec. 2023.
- [11] D. Boneh, C. Gentry, B. Lynn, and H. Shacham, "Aggregate and verifiably encrypted signatures from bilinear maps," in *Proc. Int. Conf. Theory Appl. Cryptograph. Techn.*, 2003, pp. 416–432.
- [12] H. S. Rhee, J. H. Park, W. Susilo, and D. H. Lee, "Trapdoor security in a searchable public-key encryption scheme with a designated tester," *J. Syst. Softw.*, vol. 83, no. 5, pp. 763–771, 2010.
- [13] Z. Zhang, Z. Wang, and S. Li, "Research and implementation on an efficient public key encryption algorithm with keyword search scheme," in *Proc. IEEE 5th Int. Conf. Cloud Comput. Big Data Anal. (ICCCBDA)*, 2020, pp. 314–319.
- [14] R. Zhang, R. Xue, T. Yu, and L. Liu, "PVSAE: A public verifiable searchable encryption service framework for outsourced encrypted data," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, 2016, pp. 428–435.
- [15] J. Baek, R. Safavi-Naini, and W. Susilo, "Public key encryption with keyword search revisited," in *Proc. Int. Conf. Comput. Sci. Its Appl.*, 2008, pp. 1249–1259.
- [16] Q. Huang and H. Li, "An efficient public-key searchable encryption scheme secure against inside keyword guessing attacks," *Inf. Sci.*, vols. 403–404, pp. 1–14, Sep. 2017.
- [17] B. Qin, H. Cui, X. Zheng, and D. Zheng, "Improved security model for public-key authenticated encryption with keyword search," in *Proc. 15th Int. Conf. Provable Practical Security*, 2021, pp. 19–38.
- [18] R. Chen, Y. Mu, G. Yang, F. Guo, and X. Wang, "Dual-server public-key encryption with keyword search for secure cloud storage," *IEEE Trans. Inf. Forensics Security*, vol. 11, pp. 789–798, 2015.
- [19] L. Cheng and F. Meng, "Public key authenticated encryption with keyword search from LWE," in *Proc. Eur. Symp. Res. Comput. Security*, 2022, pp. 303–324.
- [20] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proc. 35th Annu. Symp. Found. Comput. Sci.*, 1994, pp. 124–134.
- [21] X. Pan and F. Li, "Public-key authenticated encryption with keyword search achieving both multi-ciphertext and multi-trapdoor indistinguishability," *J. Syst. Archit.*, vol. 115, May 2021, Art. no. 102075.
- [22] Z.-Y. Liu, Y.-F. Tseng, R. Tso, M. Mambo, and Y.-C. Chen, "Public-key authenticated encryption with keyword search: Cryptanalysis, enhanced security, and quantum-resistant instantiation," in *Proc. ACM Asia Conf. Comput. Commun. Security*, 2022, pp. 423–436.
- [23] Z. Jiang, K. Zhang, L. Wang, and J. Ning, "Forward secure public-key authenticated encryption with conjunctive keyword search," *Comput. J.*, vol. 66, no. 9, pp. 2265–2278, 2023.
- [24] Y. Lu and J. Li, "Privacy-preserving and forward public key encryption with field-free multi-keyword search for cloud encrypted data," *IEEE Trans. Cloud Comput.*, vol. 11, no. 4, pp. 3619–3630, Oct./Dec. 2023.
- [25] M. R. Senouci, I. Benkhaddra, A. Senouci, and F. Li, "An efficient and secure certificateless searchable encryption scheme against keyword guessing attacks," *J. Syst. Archit.*, vol. 119, Oct. 2021, Art. no. 102271.
- [26] Y. Lu and J. Li, "Lightweight public key authenticated encryption with keyword search against adaptively-chosen-targets adversaries for mobile devices," *IEEE Trans. Mobile Comput.*, vol. 21, no. 12, pp. 4397–4409, Dec. 2022.
- [27] L. Chen, J. Li, J. Li, and J. Weng, "PAESS: Public-key authentication encryption with similar data search for pay-per-query," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 9910–9923, 2024.
- [28] Y. Xu, H. Cheng, J. Li, X. Liu, X. Zhang, and M. Wang, "Lightweight multi-user public-key authenticated encryption with keyword search," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 3234–3246, 2025.
- [29] C.-K. Chu, S. Chow, W.-G. Tzeng, J. Zhou, and R. H. Deng, "Key-aggregate cryptosystem for scalable data sharing in cloud storage," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 2, pp. 468–477, Feb. 2014.
- [30] S. Patranabis and D. Mukhopadhyay, "Identity-based key aggregate cryptosystem from multilinear maps," *Cryptology ePrint Arch., IACR, Bellevue, WA, USA, Rep.* 2016/693, 2016.
- [31] S. Patranabis, Y. Shrivastava, and D. Mukhopadhyay, "Dynamic key-aggregate cryptosystem on elliptic curves for online data sharing," in *Proc. 16th Int. Conf. Cryptol.*, 2015, pp. 25–44.
- [32] C. Guo et al., "Key-aggregate authentication cryptosystem for data sharing in dynamic cloud storage," *Future Gener. Comput. Syst.*, vol. 84, pp. 190–199, Jul. 2018.
- [33] J. Liu, J. Qin, X. Zhang, and H. Wang, "Efficient key-aggregate cryptosystem with user revocation for selective group data sharing in cloud storage," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 6042–6055, Nov. 2024.
- [34] Z. Liu and Y. Liu, "Verifiable and authenticated searchable encryption scheme with aggregate key in cloud storage," in *Proc. 14th Int. Conf. Comput. Intell. Security (CIS)*, 2018, pp. 421–425.
- [35] J. Lee, M. Kim, J. Oh, Y. Park, K. Park, and S. Noh, "A secure key aggregate searchable encryption with multi delegation in cloud data sharing service," *Appl. Sci.*, vol. 11, no. 19, p. 8841, 2021.
- [36] Z. Liu, T. Li, P. Li, C. Jia, and J. Li, "Verifiable searchable encryption with aggregate keys for data sharing system," *Future Gener. Comput. Syst.*, vol. 78, pp. 778–788, Jan. 2018.
- [37] X. Wang, X. Cheng, and Y. Xie, "Efficient verifiable key-aggregate keyword searchable encryption for data sharing in outsourcing storage," *IEEE Access*, vol. 8, pp. 11732–11742, 2020.
- [38] X. Wang, Y. Xie, X. Cheng, and Z. Jiang, "An efficient key-aggregate keyword searchable encryption for data sharing in cloud storage," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, 2019, pp. 1–6.
- [39] A. Kiayias, O. Oksuz, A. Russell, Q. Tang, and B. Wang, "Efficient encrypted keyword search for multi-user data sharing," in *Proc. 21st Eur. Symp. Res. Comput. Security*, Heraklion, Greece, Sep. 2016, pp. 173–195.
- [40] H. S. Trivedi and S. J. Patel, "Key-aggregate searchable encryption with multi-user authorization and keyword untraceability for distributed IoT healthcare systems," *Trans. Emerg. Telecommun. Technol.*, vol. 34, no. 4, p. e4734, 2023.
- [41] J. Li, Q. Gan, F. Wang, and X. Wang, "Provably secure key aggregate searchable encryption with verifiability for online data sharing in cloud computing," 2021. [Online]. Available: <http://dx.doi.org/10.2139/ssrn.3992940>
- [42] K. Zhang, X. Hu, J. Zhao, L. Wei, and J. Ning, "Blockchain-based revocable key-aggregate searchable encryption for group data sharing in cloud-assisted industrial IoT," *IEEE Internet Things J.*, vol. 12, no. 11, pp. 16899–16911, Jun. 2025.
- [43] J. Lee, J. Oh, D. Kwon, M. Kim, K. Kim, and Y. Park, "Blockchain-enabled key aggregate searchable encryption scheme for personal health record sharing with multidelegation," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 17482–17494, May 2024.
- [44] M. Padhya and D. C. Jinwala, "R-OO-KASE: Revocable online/offline key aggregate searchable encryption," *Data Sci. Eng.*, vol. 5, no. 4, pp. 391–418, 2020.
- [45] C. Lan, H. Li, C. Wang, X. Yang, and H. Yao, "KASE-AKA: Key-aggregate keyword searchable encryption against keyword guessing attack and authorization abuse," *Comput. Stand. Interfaces*, vol. 90, Aug. 2024, Art. no. 103852.
- [46] T. Li, Z. Liu, P. Li, C. Jia, Z. L. Jiang, and J. Li, "Verifiable searchable encryption with aggregate keys for data sharing in outsourcing storage," in *Proc. 21st Aust. Conf. Inf.*, 2016, pp. 153–169.
- [47] Y. Yao, Z. Zhai, J. Liu, and Z. Li, "Lattice-based key-aggregate (searchable) encryption in cloud storage," *IEEE Access*, vol. 7, pp. 164544–164555, 2019.



Xihui Lu received the B.S. degree in software engineering from Xiangtan University, Xiangtan, China, in 2019. She is currently pursuing the master's degree with the Department of Cryptography and Cyber Security School of Software Engineering, East China Normal University, Shanghai, China.

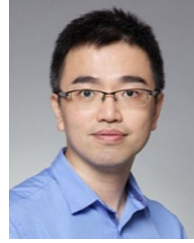
Her research interests include searchable encryption and information security.



Zhenfu Cao (Senior Member, IEEE) received the B.Sc. degree in computer science and technology and the Ph.D. degree in mathematics from Harbin Institute of Technology, Harbin, China, in 1983 and 1999, respectively.

He is currently a Distinguished Professor with East China Normal University, Shanghai, China. Since 1981, he has been published over 400 academic papers in journals or conferences. His research interests include cryptography, number theory, and information security.

Dr. Cao has received a number of awards, including the Ying-Tung Fok Young Teacher Award, in 1989, the National Outstanding Youth Fund of China, in 2002, and the Special Allowance by the State Council, in 2005. He was a co-recipient of the 2007 IEEE International Conference on Communications Computer Award, in 2007.



Jiachen Shen (Member, IEEE) received the bachelor's degree from Shanghai Jiao Tong University, Shanghai, China, in 2001, and the master's and Ph.D. degrees from the University of Louisiana at Lafayette, Lafayette, LA, USA, in 2003 and 2008, respectively.

He joined East China Normal University, Shanghai, in 2015. His research interests include applied cryptography, cloud security, searchable encryption, and blockchains.



Xiaolei Dong (Member, IEEE) received the M.S. and Ph.D. degrees in mathematics from the Harbin Institute of Technology, Harbin, China, in 1998 and 2001, respectively.

She is currently a Distinguished Professor with East China Normal University, Shanghai, China. She hosts a lot of research projects supported by the National Basic Research Program of China (973 Program), the National Natural Science Foundation of China, and the Special Funds on Information Security of the National Development and Reform

Commission. Her research interests include cryptography, number theory, and trusted computing.